

Software Engineering

Developing Requirements - Exercises -

Exercise 1

- The aim is to develop an application to handle some of the functions of a **mail-order company** that manages a warehouse of goods, takes orders from customers by telephone, and ships goods overnight to customers. It is assumed that the use case analysis for this application is completed. The result is summarized as follows:
 - The **actors** of this application are: the shipper (who sends or transports goods), the receiver, the inventory taker, the warehouse supervisor, the telephone operator, and the telephone operator supervisor.

Exercise 1

- The application should provide the following **use cases**:
 - All users can: sign in, sign out, and change the password. In addition:
 - The shipper can: display the (next) order, and display the location of an item;
 - The receiver can: enter the received order, and display the location of an item;
 - The inventory taker can: display inventory statistics, and display the location of an item;
 - The warehouse supervisor can perform all the shipper, receiver, and inventory taker use cases, plus: assign an order to shipper, assign an order to receiver.
 - The telephone operator can: answer a call, display customer info;
 - The telephone operator supervisor can perform all the telephone operator use cases, plus: display operator statistics

Exercise 1

Draw an UML use case diagram to specify the requirements of the mail order system.

Software Engineering

PRISM

Formal Modeling and Analysis

- Exercises -

Exercise 1

- Source: Dynamic Power Management (DPM) case study available from <http://www.prismmodelchecker.org/courses/pmc1112/>
<http://www.prismmodelchecker.org/tutorial/>
 - Start from (P, S and R) properties given in the course lecture slides
1. Add a reward structure to the model called "lost", which assigns 1 to every transition of the model labelled with action "request" from a state where the queue is full. Next, create a new property to check the expected cumulated reward up until time T. Don't forget to specify which reward structure you want in the property.
 2. The performance of the system is actually a trade-off between several factors. In the sequel we study the power consumption of the system, assuming the energy is used at rate 0.13, 0.95 or 2.15 for SP power states "sleep", "idle" and "busy", respectively. In addition, transitions between power states have a fixed energy cost: going from "sleep" to "idle" uses 7.0 units and from "idle" to "sleep" uses 0.067 units. Add a reward structure to the model representing the power consumption. Use a cumulative reward property to investigate the energy consumption over time of the system.

Exercise 2

The following PRISM code describes a DTMC:

```
dtmc
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 1:true;
endmodule
```

[1] The PRISM language semantics, available from:
<http://www.prismmodelchecker.org/doc/semantics.pdf>

Define the semantics of this PRISM model following [1]

- a) Construct the system module from the component modules
 - b) Give the semantics of the system module as a transition probability matrix $P:S \times S \rightarrow [0,1]$
- We recall that a (discrete) probability distribution over a countable set S is a function $\mu:S \rightarrow [0,1]$ satisfying $\sum_{s \in S} \mu(s) = 1$.
 - We use $[s_0 \rightarrow p_0, \dots, s_n \rightarrow p_n]$ to denote the distribution that chooses s_i with probability p_i (for $1 \leq i \leq n$) and $\text{Dist}(S)$ for the set of distributions over S

Exercise 3

The following PRISM code describes a CTMC:

```
ctmc
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 4.5:(v1'=0) + 0.5:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 3.5:(v2'=0) + 1.5:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 2:true;
endmodule
```

[1] The PRISM language semantics, available from:
<http://www.prismmodelchecker.org/doc/semantics.pdf>

Define the semantics of this PRISM model following [1]

- a) Construct the system module from the component modules
- b) Give the semantics of the system module as a transition rate matrix $R: S \times S \rightarrow \mathbb{R}_{\geq 0}$

Exercise 4

The following PRISM code describes a MDP:

```
mdp
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 1:true;
endmodule
```

[1] The PRISM language semantics, available from:
<http://www.prismmodelchecker.org/doc/semantics.pdf>

Define the semantics of this PRISM model following [1]

- Construct the system module from the component modules
- Give the semantics of the system module as the function (given in [1]):

$$\text{Steps} : S \rightarrow 2^{\text{Dist}(S)}$$

- S is the set of states, $2^{\text{Dist}(S)}$ = power set of $\text{Dist}(S)$, i.e., the set of all subsets of $\text{Dist}(S)$
- Give the semantics of the system module when $\text{Steps} : S \rightarrow 2^{(\text{Act} \cup \{\uparrow\}) \times \text{Dist}(S)}$ is (re)defined as follows:

$$\text{Steps}(s) = \{(\alpha, \mu_{c,s}) \mid c \in C, s \in S\}$$

- $(\alpha \in) \text{Act}$ = set of action labels, $\uparrow \notin \text{Act}$,
- $\alpha = a$, if $c = [a] g \rightarrow \lambda_1 : u_1 + \dots + \lambda_n : u_n$
- $\alpha = \uparrow$, if $c = [] g \rightarrow \lambda_1 : u_1 + \dots + \lambda_n : u_n$
- $\mu_{c,s} : \text{Dist}(S)$ - $\mu_{c,s}$ are given in [1]

mdp

// Create system module

// anonymous or non synchronized => copied to M

// synchronized => [a] g1 & g2 => p1*p2 : u1 & u2 + ...

module M

 v1 : [0..1] init 0;

 v2 : [0..1] init 0;

 [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1); //c1

 [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1); //c2

 [a] v1=0 & v2=1 & v1=0 & v2=1 -> 1 * 1:(v1'=1) & true; //c3, true = (v2'=v2)

 [b] v1=1 & v1=1 -> 1 * 1:true & true; //c4, true = (v1'=v1), true = (v2'=v2)

endmodule

// Set of states

S = { s0, s1, s2, s3 }

s0 = (0, 0) // v1 = 0, v2 = 0

s1 = (0,1) // v1 = 0, v2 = 1

s2 = (1,0) // v1 = 1, v2 = 0

s3 = (1,1) // v1 = 1, v2 = 1

// Set of states where the commands can take place

Sc1 = {s0}

$Sc2 = \{s0\}$

$Sc3 = \{s1\}$

$Sc4 = \{s3, s2\}$

MDP=(S, s0, Steps, L), L left unspecified in this example

// Construct step function Steps

$\mu_{c1,s0}(s0) = 0.9, \mu_{c1,s0}(s2) = 0.1 \Rightarrow \mu_{c1,s0} = [s0 \rightarrow 0.9, s2 \rightarrow 0.1]$

$\mu_{c2,s0}(s0) = 0.7, \mu_{c2,s0}(s1) = 0.3 \Rightarrow \mu_{c2,s0} = [s0 \rightarrow 0.7, s1 \rightarrow 0.3]$

$\mu_{c3,s1}(s3) = 1 \Rightarrow \mu_{c3,s1} = [s3 \rightarrow 1]$

$\mu_{c4,s2}(s2) = 1 \Rightarrow \mu_{c4,s2} = [s2 \rightarrow 1]$

$\mu_{c4,s3}(s3) = 1 \Rightarrow \mu_{c4,s3} = [s3 \rightarrow 1]$

Steps (s0) = { $\mu_{c1,s0}, \mu_{c2,s0}$ } = { [s0 → 0.9, s2 → 0.1], [s0 → 0.7, s1 → 0.3] }

Steps (s1) = { $\mu_{c3,s1}$ } = { [s3 → 1] }

Steps (s2) = { $\mu_{c4,s2}$ } = { [s2 → 1] }

Steps (s3) = { $\mu_{c4,s3}$ } = { [s3 → 1] }

Steps (s0) = { (↑, $\mu_{c1,s0}$), (↑, $\mu_{c2,s0}$) } = { (↑, [s0 → 0.9, s2 → 0.1]), (↑, [s0 → 0.7, s1 → 0.3]) }

Steps (s1) = { (a, $\mu_{c3,s1}$) } = { (a, [s3 → 1]) }

Steps (s2) = { (b, $\mu_{c4,s2}$) } = { (b, [s2 → 1]) }

Steps (s3) = { (b, $\mu_{c4,s3}$) } = { (b, [s3 → 1]) }

ctmc

// Create system module

// anonymous or non synchronized => copied to M

// synchronized => [a] g1 & g2 => p1*p2 : u1 & u2 + ...

module M

 v1 : [0..1] init 0;

 v2 : [0..1] init 0;

 [] v1=0 & v2=0 -> 4.5:(v1'=0) + 0.5:(v1'=1); //c1

 [] v1=0 & v2=0 -> 3.5:(v2'=0) + 1.5:(v2'=1); //c2

 [a] v1=0 & v2=1 & v1=0 & v2=1 -> 1:(v1'=1)&true; //c3, true means (v2'=v2)

 [b] v1=1 & v1=1 -> 2:true & true; //c4, true = (v1'=v1), true = (v2'=v2)

endmodule

// Set of states

S = { s0, s1, s2, s3 }

s0 = (0, 0) // v1 = 0, v2 = 0

s1 = (0,1) // v1 = 0, v2 = 1

s2 = (1,0) // v1 = 1, v2 = 0

s3 = (1,1) // v1 = 1, v2 = 1

// Set of states where the commands can take place

Sc1 = {s0}

$S_{c2} = \{s_0\}$

$S_{c3} = \{s_1\}$

$S_{c4} = \{s_3, s_2\}$

CTMC=(S, s0, R, L), L left unspecified in this example

// Construct R = rate transition matrix

$\mu_{c1,s0}(s_0) = 4.5, \mu_{c1,s0}(s_2) = 0.5$

$\mu_{c2,s0}(s_0) = 3.5, \mu_{c2,s0}(s_1) = 1.5$

$\mu_{c3,s1}(s_3) = 1$

$\mu_{c4,s2}(s_2) = 2$

$\mu_{c4,s3}(s_3) = 2$

	s0	s1	s2	s3
s0	8	1.5	0.5	0
s1	0	0	0	1
s2	0	0	2	0
s3	0	0	0	2

s0

s1

s2

s3

=R

	s0	s1	s2	s3
s0	0.8	0.15	0.05	0
s1	0	0	0	1
s2	0	0	1	0
s3	0	0	0	1

s0

s1

s2

s3

=P

// Stochastic matrix

dtmc

// Create system module

// anonymous or non synchronized => copied to M

// synchronized => [a] g1 & g2 => p1*p2 : u1 & u2 + ...

module M

 v1 : [0..1] init 0;

 v2 : [0..1] init 0;

 [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1); //c1

 [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1); //c2

 [a] v1=0 & v2=1 & v1=0 & v2=1 -> 1 * 1:(v1'=1) & true; //c3, true = (v2'=v2)

 [b] v1=1 & v1=1 -> 1 * 1:true & true; //c4, true = (v1'=v1), true = (v2'=v2)

endmodule

// Set of states

S = { s0, s1, s2, s3 }

s0 = (0, 0) // v1 = 0, v2 = 0

s1 = (0,1) // v1 = 0, v2 = 1

s2 = (1,0) // v1 = 1, v2 = 0

s3 = (1,1) // v1 = 1, v2 = 1

// Set of states where the commands can take place

Sc1 = {s0}

$S_{c2} = \{s_0\}$

$S_{c3} = \{s_1\}$

$S_{c4} = \{s_3, s_2\}$

DTMC=(S, s0, P, L), L left unspecified in this example

// Construct P = probability transition matrix

$\mu_{c1,s0}(s_0) = 0.9, \mu_{c1,s0}(s_2) = 0.1 \Rightarrow \mu_{c1,s0} = [s_0 \rightarrow 0.9, s_2 \rightarrow 0.1]$

$\mu_{c2,s0}(s_0) = 0.7, \mu_{c2,s0}(s_1) = 0.3 \Rightarrow \mu_{c2,s0} = [s_0 \rightarrow 0.7, s_1 \rightarrow 0.3]$

$\mu_{c3,s1}(s_3) = 1 \Rightarrow \mu_{c3,s1} = [s_3 \rightarrow 1]$

$\mu_{c4,s2}(s_2) = 1 \Rightarrow \mu_{c4,s2} = [s_2 \rightarrow 1]$

$\mu_{c4,s3}(s_3) = 1 \Rightarrow \mu_{c4,s3} = [s_3 \rightarrow 1]$

//not normalized transition matrix:

	s0	s1	s2	s3
s0	1.6	0.3	0.1	0
s1	0	0	0	1
s2	0	0	1	0
s3	0	0	0	1

= $\bar{P}$

	s0	s1	s2	s3	
s0	0.8	0.15	0.05	0	
s1	0	0	0	1	
s2	0	0	1	0	
s3	0	0	0	1	=P

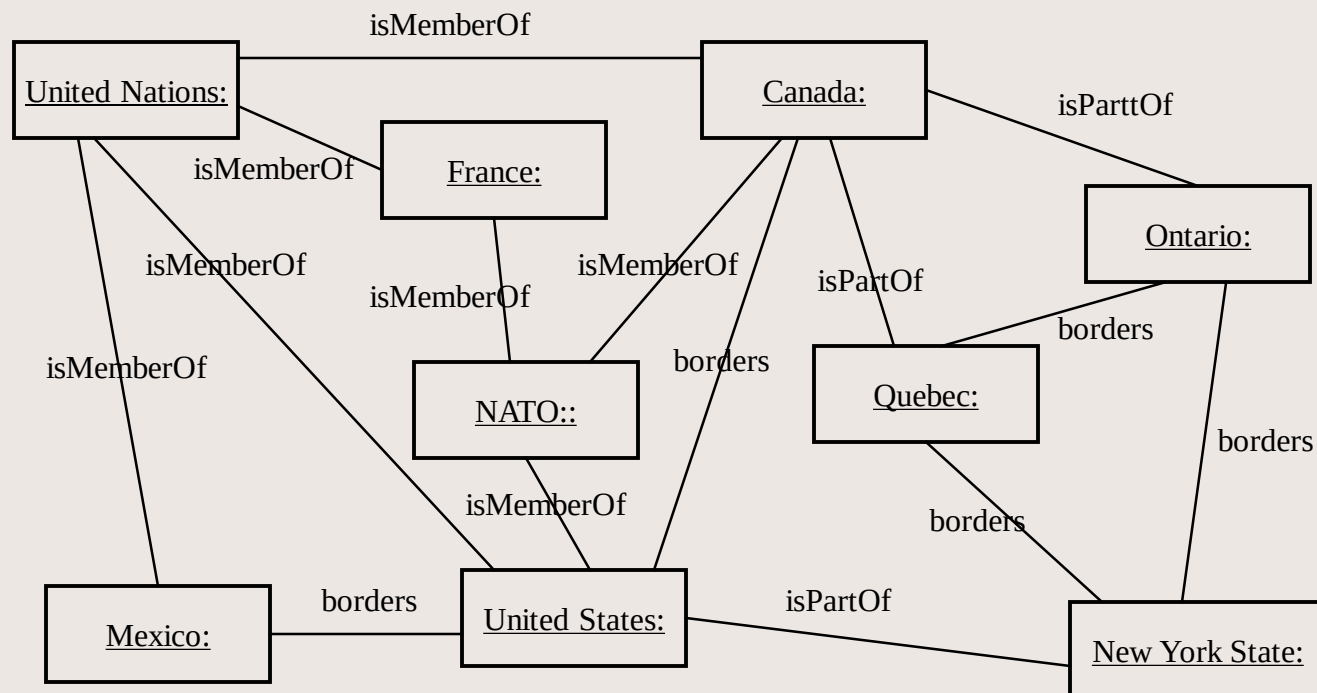
// Stochastic matrix

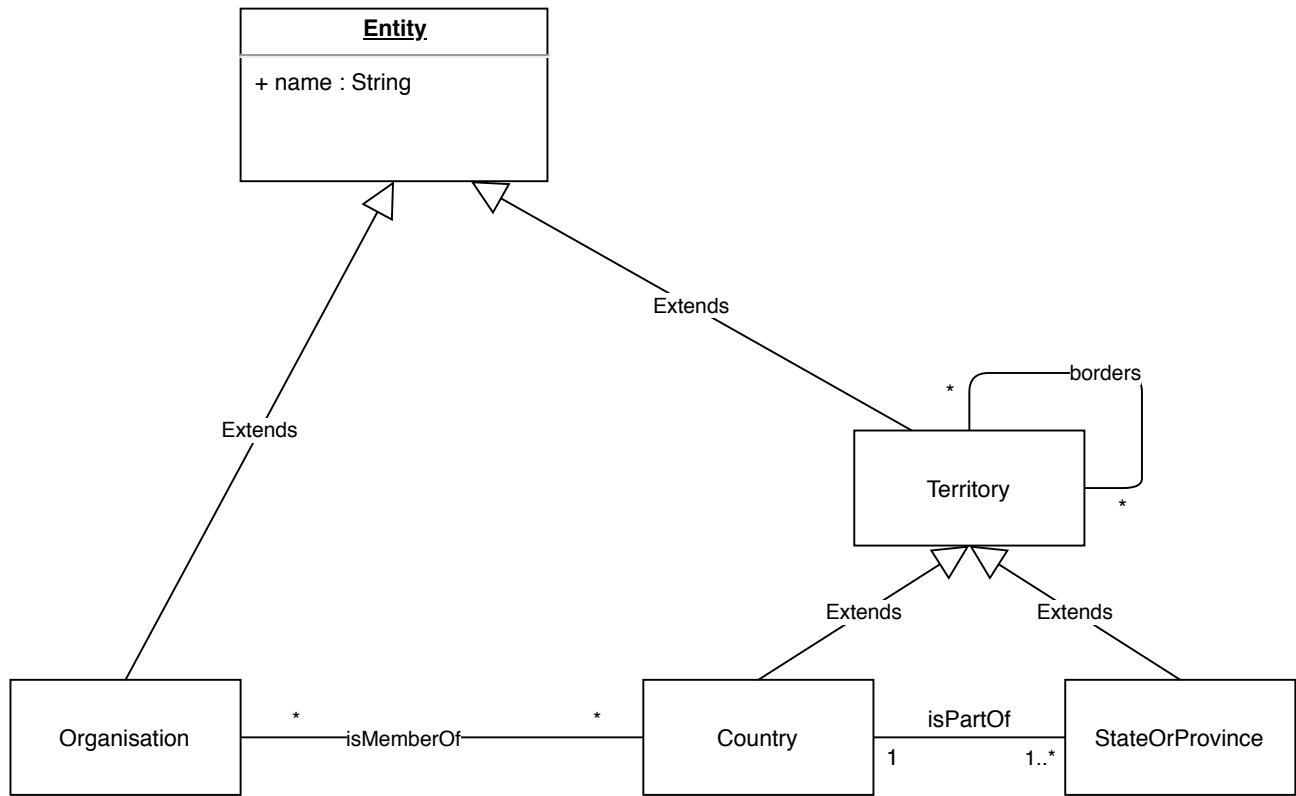
Software Engineering

Modeling with classes - Exercises -

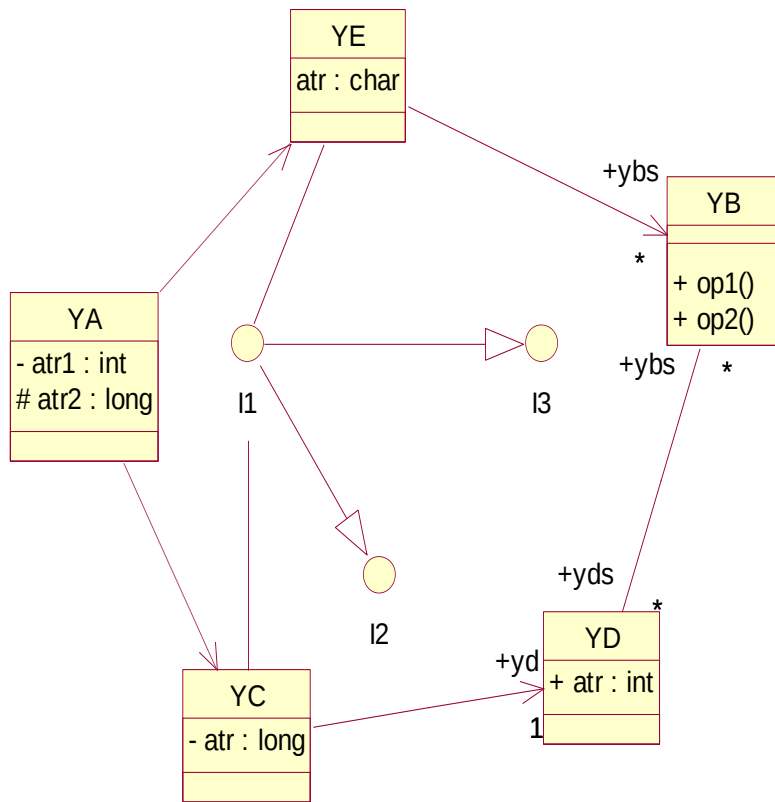
Exercise 3

- (E91, [LL05]) Draw a class diagram that could generate the object diagram shown below (figure 5.18, [LL05]). Make reasonable choices for multiplicity.





Forward engineering (UML -> Java)



```
public interface I2 {  
}  
  
public interface I3 {  
}  
  
public interface I1 extends I2, I3 {  
}  
  
public class YA {  
    private int atr1;  
    protected long atr2;  
}  
  
public class YB {  
    public YD[] yds;  
  
    public op1() {  
    }  
    public op2() {  
    }  
}  
  
public class YC implements I1 {  
    private long atr;  
    public YD yd;  
}  
  
public class YD {  
    public int atr;  
    public YB[] ybs;  
}  
  
public class YE implements I1 {  
    char atr;  
    public YB[] ybs;  
}
```

Reverse engineering (Java -> UML)

```
class F extends G implements I3 {
    private K[] ks;
    int i;
    public K k;
    protected H h;
    F() {}
}

class G {
    public G() {
    }
    public void f(H h) {}
}

class H {
    public H() {
    }
}

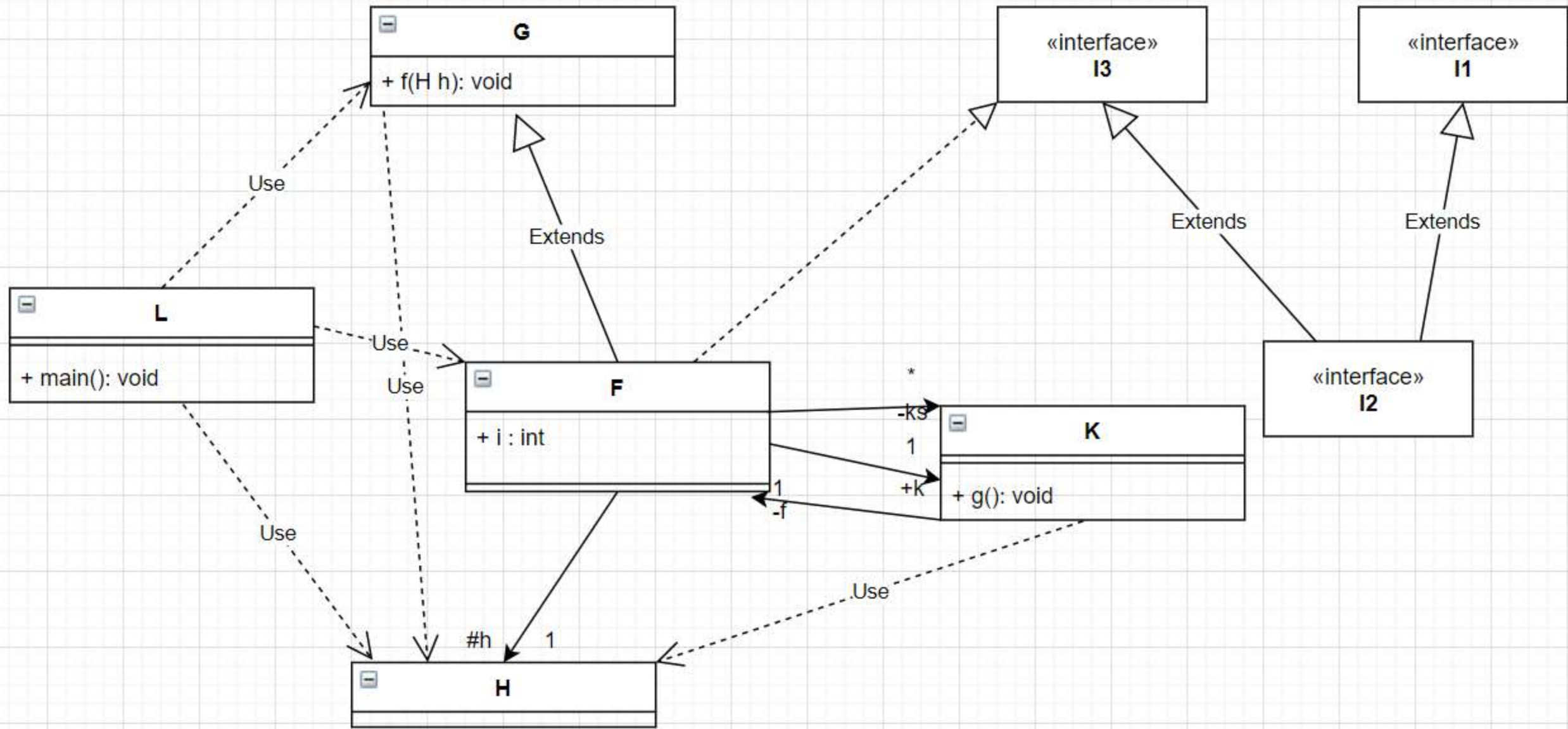
class K {
    private F f;
    public K() {
    }
    void g() { H h = new H(); }
}

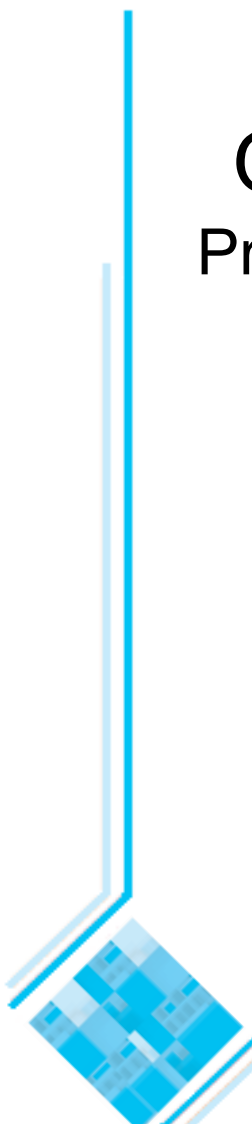
class L {
    public static void main(String args[]) {
        H h = new H();
        G g = new G();
        F f = new F();
        System.out.println("Test");
    }
}

interface I1 {
}

interface I2 extends I1, I3 {
}

interface I3 {
}
```



Object-Oriented Software Engineering

Practical Software Development using UML and Java

Chapter 6: Using Design Patterns

- Exercises -

References:

[LL05] T. Lethbridge, R. Laganier. *Object Oriented Software Engineering Practical Software Development using UML and Java* (Second Edition). McGraw Hill, 2005.

Exercises: E113 ([LL05] page 225), E116 ([LL05] page 228)

[KNP07] M. Kwiatowska, G. Norman, and D. Parker, Stochastic Model Checking, *Lecture Notes in Computer Science*, vol. 4486, pp. 53-113, 2007.

Exercise 1

Exercise: Use the Composite (General Hierarchy) design pattern to model the Lisp data type, which can be specified using ML/Haskell algebraic data types as follows:

```
data Lisp = Nil | Number Int | Symbol String | Cons Lisp Lisp
```

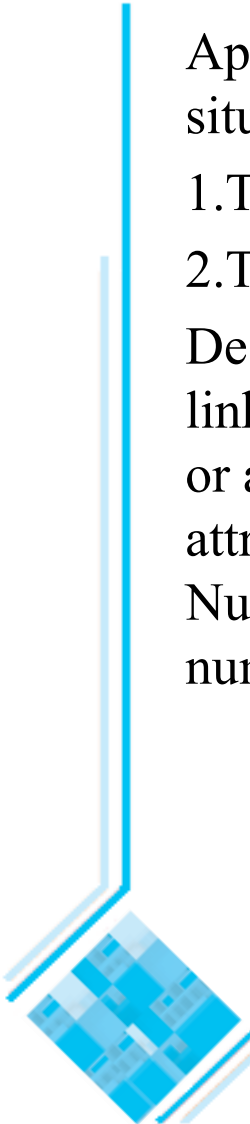


Exercise 2: (source: E113, [LL05])

Apply the Abstraction–Occurrence design pattern in the following situations :

- 1.The issues of a periodical
- 2.The copies of the issues of a periodical

Design a class diagram showing the association and the attributes of the two linked classes. You can assume that the periodical (e.g., a scientific journal or a magazine) with its issues and copies are described by the following attributes: name (of the periodical), ISSN (International Standard Serial Number), volume number, issue number, publication date and bar code number.



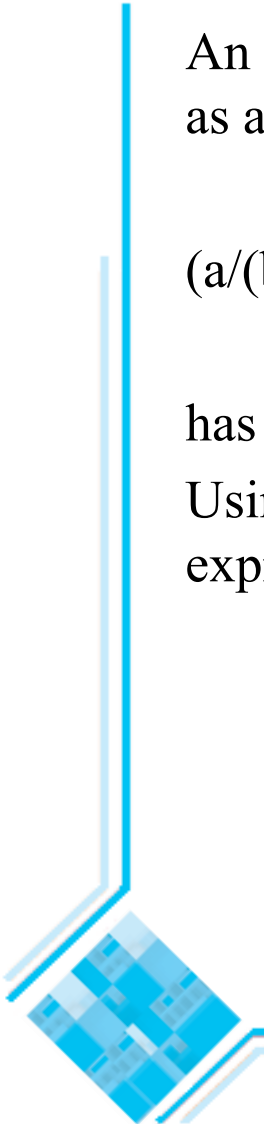
Exercise 3: (source: E116, [LL05])

An expression in Java (or some other programming language) can be seen as a hierarchy of sub-expressions. For example,

$(a/(b+c)) + (b - (\text{func}(d) * (e/(f+g))))$

has $a/(b+c)$ as an (immediate) sub-expression.

Using an appropriate design pattern create a class diagram to represent expressions in Java.

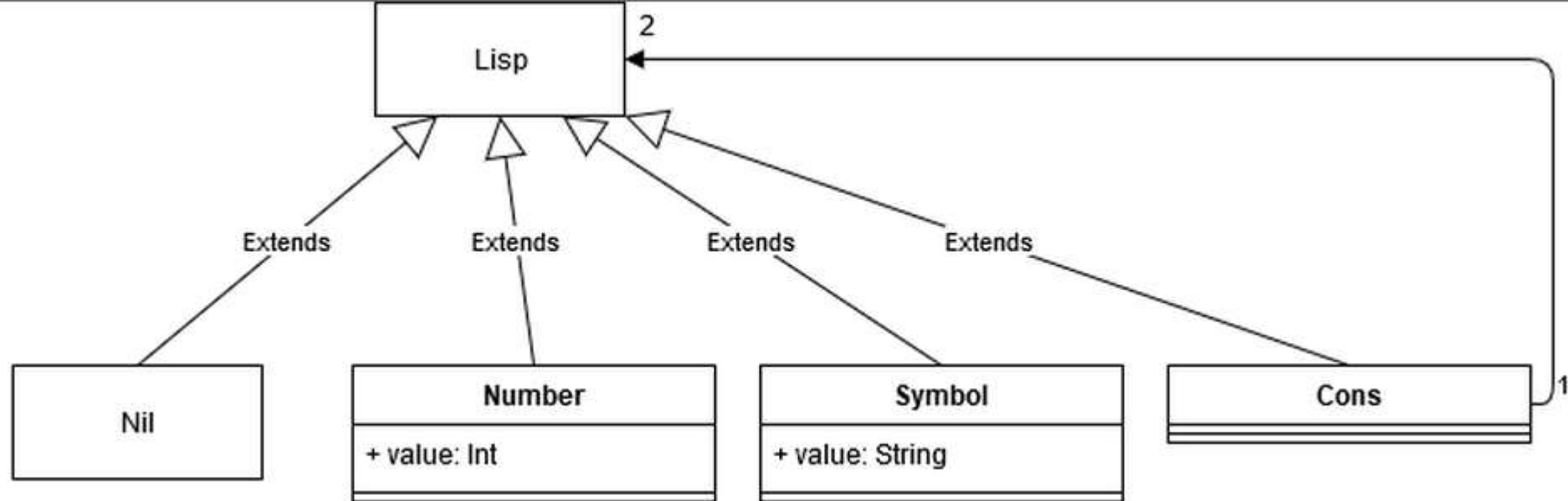


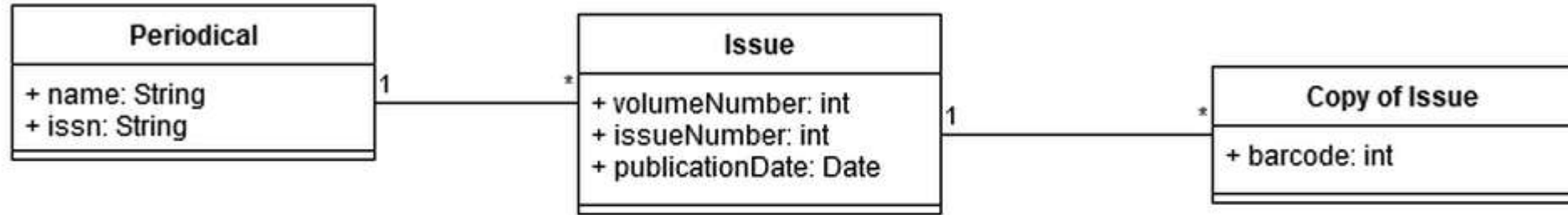
Exercise 4: Using General Hierarchy pattern to describe temporal logic syntax

Using the General Hierarchy (or Composite) pattern create a class diagram to represent PCTL (Probabilistic Computation Tree Logic) expressions. PCTL is a probabilistic extension of the temporal logic CTL. The syntax of PCTL is given below in BNF:

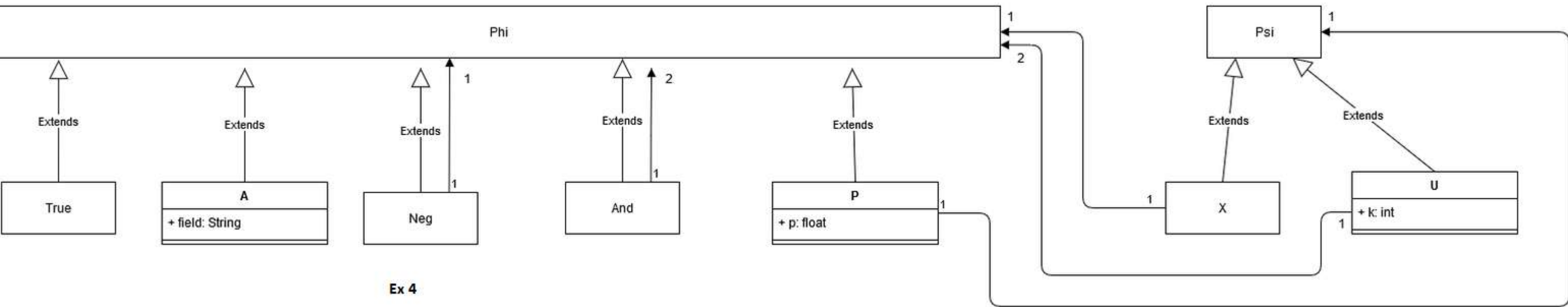
$$\Phi ::= \text{true} \mid a \mid \neg \Phi \mid \Phi \wedge \Phi \mid P_{\sim p}[\Psi]$$
$$\Psi ::= X \Phi \mid \Phi \text{ U}_{\leq k} \Phi$$

Ex 1





Ex 2



Software Engineering

Modeling
interactions and behavior
- Exercises -

References

J.C.M. Baeten, W.P. Weijland, *Process Algebra*, Cambridge University Press, 1990.

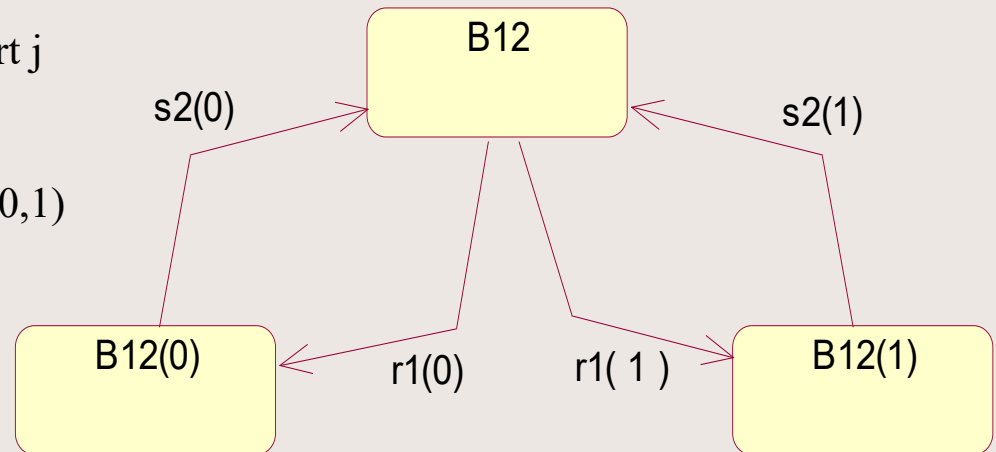
Exercise 1

- Consider a **bit buffer** B12 of **capacity one** with ports 1 (input) and 2 (output):



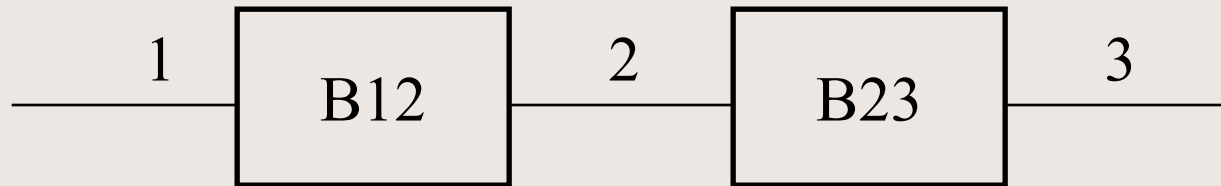
- The behavior of B12 is described by the following State Transition Diagram (STD), where:

- $rj(b)$ = receive bit b at port j
- $sj(b)$ = send bit b at port j
- $B12$ = buffer empty
- $B12(b)$ = buffer full ($b = 0,1$)

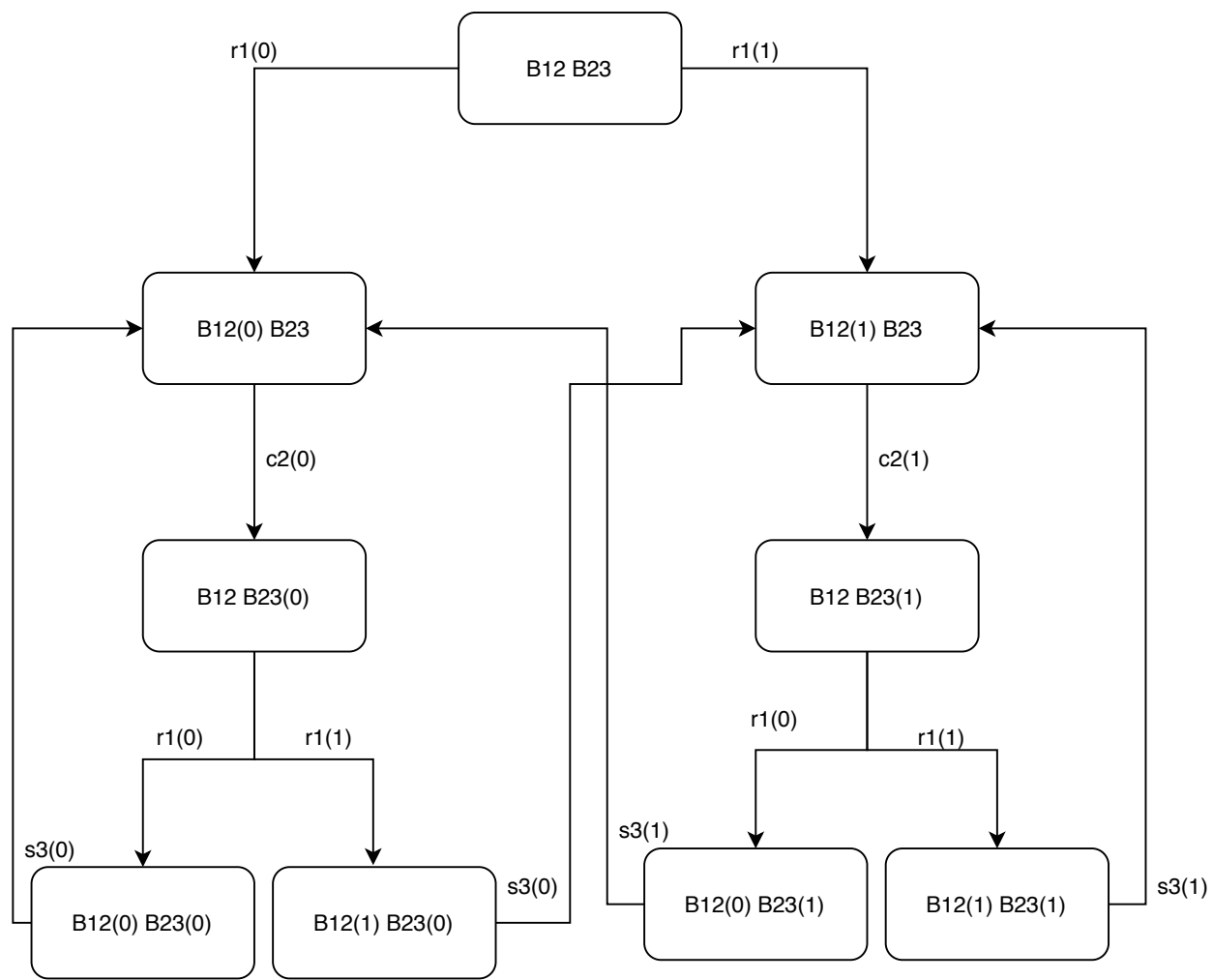


Exercise 1

- Draw a STD to model the behavior of two buffers B12 and B23 connected at port 2:
B12 can send data at port 2, this data is received by B23.
 - By convention, the simultaneous execution of a pair of actions $sj(b)$ (send bit b at port j) and $rj(b)$ (receive bit b at port j) results in a transmission of b by a communication at port j : $cj(b)$



- Use the resulted STD to show that two one bit buffers connected in this way behave as a buffer of capacity two.



Software Engineering

Architecting and Designing
Software, Testing and Inspecting
to Ensure High Quality

- Exercises –

References

[LL05] T. Lethbridge, R. Laganriere. *Object Oriented Software Engineering Practical Software Development using UML and Java* (Second Edition). McGraw Hill, 2005.
Exercises: E172 ([LL05] pg 320), E176 ([LL05] pg 328)

Exercise 1

A software design should lead to modules that exhibit high cohesion and low coupling (Source: E172, [LL05]):

1. Categorize the following aspect(s) of a design by the types of cohesion (functional, layer, communicational, sequential, procedural, temporal, utility) that it (they) would exhibit if properly designed:
 - a. All the information concerning bookings is kept inside a particular class, and everything else is kept out
 - b. A separate module (subsystem) is created that runs every night to generate statistics about the previous day's sales.
2. Categorize the following aspect of a design by the type(s) of coupling (content, common, control, stamp, data, routine call, type use, import, external) it exhibits (Source: E176, [LL05]) :
 1. A system has a class called Address. This class has four public variables constituting different parts of an address. Several different classes, such as Person and Airport manipulate instances of this class, directly modifying the fields of addresses. Also, many methods declare one of their arguments to be an Address.

Software Engineering

Declarative prototyping - Exercises -

Exercise 1

- Quicksort with difference lists – declarative prototype:

```
qsd :: [Int] -> [Int] -> [Int]
qsd []      ys = ys
qsd (x : xs) ys =
    let (ls,gs) = partit x xs
    in qsd ls (x : qsd gs ys)
```

```
partit :: Int -> [Int] -> ([Int],[Int])
partit p []      = ([],[ ])
partit p (x:xs) =
    let (ls,gs) = partit p xs
    in  if (x < p) then (x : ls,gs) else (ls,x:gs)
```

```
Main> qsd [1,7,3,12,4,2,5] [0,0,0]
[1,2,3,4,5,7,12,0,0,0]
```

Exercise 2

- Binary search tree merging using difference lists:

```
data Tree = NIL | T Tree Int Tree
flatt :: Tree -> [Int] -> [Int]
flatt NIL      ys = ys
flatt (T l n r) ys = flatt l (n : flatt r ys)

merge :: Tree -> Tree -> [Int] -> [Int]
merge NIL t ys = flatt t ys
merge t NIL ys = flatt t ys
merge (T NIL n1 r1) (T NIL n2 r2) ys =
    if (n1 < n2) then n1:merge r1 (T NIL n2 r2) ys
                  else n2:merge (T NIL n1 r1) r2 ys
merge (T (T l11 ln1 lr1) n1 r1) t2 ys =
    merge (T l11 ln1 (T lr1 n1 r1)) t2 ys
merge t1 (T (T l12 ln2 lr2) n2 r2) ys =
    merge t1 (T l12 ln2 (T lr2 n2 r2)) ys

t1 = T (T NIL 1 (T NIL 3 NIL)) 5 (T NIL 7 NIL)
t2 = T (T NIL 2 NIL) 4 (T (T NIL 6 NIL) 8 NIL)
Main> merge t1 t2 [0,0]
[1,2,3,4,5,6,7,8,0,0,0]
```

Exercise 3

- Lisp-like structure flattening – declarative prototype:

```
data Lisp = Nil | Atom Int | Cons Lisp Lisp
```

```
flat :: Lisp -> [Int] -> [Int]
```

```
flat Nil ys = ys
```

```
flat (Atom n) ys = n:ys
```

```
flat (Cons car cdr) ys = flat car (flat cdr ys)
```

```
lisp :: Lisp
```

```
lisp = Cons (Cons (Cons (Atom 1) Nil)  
              (Cons Nil (Atom 2)))  
          (Cons (Atom 3) (Atom 4))
```

```
Main> flat lisp [0,0,0]
```

```
[1,2,3,4,0,0,0]
```

```
#include <iostream>
```

```
typedef struct elem {  
    int info;  
    struct elem* next;  
} ELEM, *LIST;
```

Ex 1

```
void partit(int p, LIST xs, LIST* ls, LIST* gs) {  
    if(xs == NULL) {  
        *ls = NULL;  
        *gs = NULL;  
    } else {  
        partit(p, xs->next, ls, gs);  
  
        if(xs->info < p) {  
            xs->next = *ls;  
            *ls = xs;  
        } else {  
            xs->next = *gs;  
            *gs = xs;  
        }  
    }  
}
```

```
LIST qsd(LIST xs, LIST ys) {  
    if(xs == NULL) {  
        return ys;  
    } else {  
        LIST ls, gs;  
        partit(xs->info, xs->next, &ls, &gs);  
        xs->next = qsd(gs, ys);  
        return qsd(ls, xs);  
    }  
}
```

```

void qsd_r(LIST xs, LIST* res, LIST ys) {
    if(xs == NULL) {
        *res = ys;
    } else {
        LIST ls, gs;
        partit(xs->info, xs->next, &ls, &gs);
        qsd_r(gs, &(xs->next), ys);
        qsd_r(ls, res, xs);
    }
}

```

```

void printList(LIST ls) {
    while(ls != NULL) {
        std::cout << ls->info << " ";
        ls = ls->next;
    }
}

```

```

int main() {
    ELEM e1 {1, NULL};
    ELEM e2 {7, &e1};
    ELEM e3 {3, &e2};
    ELEM e4 {12, &e3};
    ELEM e5 {4, &e4};

    ELEM y1 {0, NULL};

    // printList(qsd(&e5, NULL));
    // printList(qsd(&e5, &y1));

    LIST res;
    // qsd_r(&e5, &res, NULL);
    qsd_r(&e5, &res, &y1);
    printList(res);
    return 0;
}

```